Characterising Global Platforms: Centralised, Decentralised, Federated, and Grassroots

Ehud Shapiro
London School of Economics and Weizmann Institute of Science

arXiv:2511.03286v5 [cs.DC] 30 Aug 2026

Abstract

Global digital platforms are distributed systems designed to serve entire populations, with some already serving billions of people. Here we propose atomic transactions-based multiagent transition systems and protocols as a formal framework to study them; introduce essential agents—minimal sets of agents the removal of which makes communication impossible; and show that the cardinality of essential agents partitions all global platforms into four classes:

1. Centralised (Facebook) – one (the server)
2. Decentralised (Bitcoin) – finite > 1 (bootstrap nodes)
3. Federated (Mastodon) – infinite but not universal (all servers)
4. Grassroots (Scuttlebutt) – universal (all agents but one)

Our illustrative formal example is a global social network, for which we provide centralised, decentralised, federated, and grassroots specifications via multiagent atomic transactions, and prove they all satisfy the same basic correctness properties, yet have different sets of essential agents as expected. We discuss informally additional global platforms—currencies, "sharing economy" apps, AI, and more.

While this may be the first formal characterisation of centralised, decentralised, and federated global platforms, grassroots platforms have two previous definitions. We prove that (1) the two definitions are incomparable; (2) each implies that all agents but one are essential; and (3) both implications are strict, presenting two platforms with all agents but one essential that are not grassroots under either definition.

This work provides the first mathematical framework for classifying any global platform—existing or imagined—by providing a multiagent atomic-transactions specification of it and determining the cardinality of the minimal set of essential agents in the ensuing multiagent protocol. It thus provides a unifying mathematical approach for the study of global digital platforms, perhaps the most important class of distributed systems today.

2012 ACM Subject Classification Theory of computation → Distributed computing models; Theory of computation → Concurrency; Computer systems organization → Peer-to-peer architectures

Keywords and phrases Global Platforms, Essential Agents, Multiagent Transition Systems, Atomic Transactions, Grassroots Protocols, Centralised, Decentralised, Federated

This paper is not eligible for the best student paper award.

Class	Canonical Example	Essential Agents	Cardinality
Centralised	Facebook	The server	One
Decentralised	Bitcoin [40]	Bootstrap nodes	Finite, > 1
Federated	Mastodon [47]	All servers/all clients	Infinite, not universal
Grassroots	Scuttlebutt [59]	All agents but one	Universal

Table 1 Classes of global platforms by cardinality of essential agents

1 Introduction

Global digital platforms are software systems that aim to serve entire populations—all of humanity in some, entire nations in others. Global platforms have emerged as humanity's primary infrastructure for social interaction, economic exchange, and information flow. Despite their ubiquity and profound influence on billions of lives, we lack a mathematical framework to study their fundamental architectures. The informal classification of platforms as centralised, decentralised, federated, or peer-to-peer is well-known, but hitherto has been imprecise: the boundaries between classes are vague, the classification is not exhaustive, and it is unclear whether the classes are mutually exclusive.

Our Framework. We propose atomic transactions-based multiagent transition systems and protocols [57] as a formal framework to study and characterize global platforms. In this framework, platforms are modelled as sets of agents whose interactions are governed by atomic transactions—indivisible state changes that can involve multiple participants.

To demonstrate the viability of this framework for specifying platforms of all four classes, we use a Facebook/X-like social network—conceived as centralised platforms—as our running example, providing atomic-transactions specifications of centralised, decentralised, federated, and grassroots social networks with feeds and followers.

Essential Agents: A Lens for Classification. We then introduce *essential agents*—minimal sets of agents the removal of which makes communication impossible (only unary transitions may occur). This concept provides a mathematical lens for understanding platform dependencies. The cardinality of essential agents in multiagent protocols provides the first formal characterisation of global platforms, partitioning them into four classes, as shown in Table 1.

Global Platforms in Practice. We review the "common knowledge" on global platforms of the four classes, exposing their fundamental importance and influence. See also Table 2.[1]

1. **Centralised platforms** employ the centralised client-server architecture, where corporations control cloud servers that mediate all interactions among people who use them. Five companies command platforms with over one billion users each: Google/Alphabet (5Bn users; \$3Trn market cap), Meta (3.8Bn; \$2Trn), ByteDance (2Bn; \$350Bn), Apple (1.5Bn; \$4Trn), and Tencent (1.4Bn; \$750Bn). Collectively, these companies have over \$10Trn market value and reach over 5Bn people. Their governance is *autocratic*: one entity holds all decision-making power, resulting in a handful of people controlling together a substantial portion of the digital lives of more than half of humanity. Facebook, Twitter, Uber, and traditional web services exemplify this architecture, where a server with a designated identity (e.g. facebook.com) remains permanently essential for platform operation. This

[1] All numbers in the paper are approximate, as of November 2025; compiled from public reporting and industry trackers and cross-verified by AI.

Class	Control	Examples
Centralised	**Corporate**	Google Search (5Bn), YouTube (2.5Bn), Gmail (2.5Bn), Facebook (3Bn), Instagram (3Bn), WhatsApp (3Bn), Messenger (1Bn), TikTok (1.6Bn), Douyin (750M), iPhone (1.5Bn), WeChat (1.4Bn), X/Twitter (500M), Discord (200M) [20], Web servers, Database servers
Decentralised	**Capital**	Bitcoin ($2.3Trn) [40], Ethereum ($500Bn) [13, 64], Tether ($180Bn), BNB ($160Bn), Solana ($110Bn), (IPFS [7], DHT/Kademlia [38], BitTorrent [17])
Federated	**Local Operators**	Mastodon (15M) [46], Matrix (80M) [37], ActivityPub networks, Email (SMTP servers)
Grassroots	**Participants**	Scuttlebutt [59] Grassroots social networks* [54], Grassroots coins and bonds* [52, 31, 56], Grassroots federations* [58], Grassroots Logic Programs* [55], * Mathematically specified, yet to be implemented

Table 2 Classification of global platforms, with user counts and market valuations where available.

architecture enables what may be characterized as *corporate control* [65, 66].

2. **Decentralised platforms** include global cryptocurrency platforms that employ the blockchain architecture, where distributed networks of nodes maintain replicated ledgers through consensus protocols while users access services through wallets and decentralised applications. Bitcoin [40] ($2.3Trn market cap) and Ethereum [13, 64] ($500Bn) exemplify this architecture. The top five platforms collectively exceed $3Trn in market value. While envisioned as decentralised alternatives to centralised financial systems, in practice cryptocurrencies exhibit extreme concentration of both ownership and control: Regarding ownership, less than 5% of cryptocurrency holders, which is less than 0.25% of the human population, own more than 90% of their value, exacerbating an already-unprecedented concentration of wealth. Regarding control, it is not only *plutocratic*, but also far from being decentralised: In Bitcoin the top two mining pools control 55% of hashrate, and in Ethereum the top four operators control 50% of staked ETH. In both mechanisms, control is realized via capital—computational resources (PoW) or staked assets (PoS). We may term this *capital control* [14]. Decentralised platforms also include IPFS [7] and Kademlia-based distributed hash tables [38], which have a finite number of boot nodes that new participants must reach to join the network. BitTorrent [17] is decentralised: its original version relies on a tracker and its trackerless version on DHT boot nodes, so in both, joining a swarm requires reaching a finite set of coordinating agents.
3. **Federated platforms** employ a distributed server architecture where multiple independent servers interoperate through shared protocols, with users attached to their chosen home server. Examples include Mastodon [46] (15M users across 10,000 servers), Matrix [37] (80M users), and the broader ActivityPub ecosystem. While being more distributed than centralised platforms, federated systems maintain a fundamental architectural distinction between servers and clients. Users cannot function without their home server, and server operators exercise control over their domains—deciding moderation policies, user access, and federation relationships. This may be characterized as *control by local operators*.
4. **Grassroots platforms** aim to provide an *egalitarian, cooperative and democratic* alternative to centralised/autocratic and decentralised/plutocratic global platforms [53, 57]. Unlike the global platforms that may only have a single instance (one Facebook, one Bitcoin), grassroots platforms enable any agent to initiate an instance, have multiple

instances operate independently, and have instances coalesce, possibly (but not necessarily) resulting in a single global instance. Grassroots platforms were specified for social networks [54], coins and bonds [52, 31, 56] and democratic federations [58, 31]. Existing grassroots platforms include Scuttlebutt [59]

Contributions. Our introduction of essential agents as a mathematically-founded notion provides two fundamental contributions to the study of global platforms. The cardinality of minimal sets of essential agents yields a comprehensive and mutually exclusive characterisation of all four known platform classes—centralised (one), decentralised (finite, > 1), federated (infinite, but not universal), and grassroots (universal). This is the first formal characterisation of centralized, decentralized, and federated global platforms, now unified with grassroots computing within a single mathematical framework. Grassroots platforms have two previous definitions. We prove that (1) the two definitions are incomparable; (2) each implies that all agents but one are essential; and (3) both implications are strict, presenting two platforms with all agents but one essential that are not grassroots under either definition.

What the formalism reveals. The framework exposes properties of global platforms that are not apparent from informal descriptions. First, it shows that social networks do not require consensus: the grassroots social network specification satisfies the same correctness properties as the centralised, decentralised, and federated specifications, yet involves no consensus mechanism, exposing a fundamental architectural mismatch in blockchain-based social networks. Second, it makes precise the distinction between "decentralised" and "grassroots," which are often conflated informally: the cardinality gap between finite and universal essential agents is sharp.

Paper organization. Section 2 presents the mathematical framework: atomic transactions, multiagent transition systems, protocols, liveness, essential agents, and the four platform classes. Section 3 uses social networks as a running example, providing transaction-based specifications for all four platform classes, stating correctness and cardinality of essential agents for each. Section 4 presents two formal definitions of grassroots protocols, proves them incomparable, and proves that both imply all agents but one are essential and that both implications are strict. Section 5 reviews related work and Section 6 concludes. Proofs of correctness and cardinality for the social network specifications appear in Appendix A, and Appendix B reviews additional global platforms of all classes.

2 Mathematical Background

Here, we recall multiagent transition systems [51], the notion of grassroots protocols and platforms [53], and their specification via multiagent atomic transactions [57], and introduce the novel notions of transaction equivalence, liveness, and essential agents, and use the cardinality of essential agents to classify global platforms.

2.1 Atomic Transactions

We assume a potentially infinite set of agents Π, but consider only finite subsets of it, so when referring to a particular set of agents $P \subset \Pi$ we assume P to be nonempty and finite. We use $\subset$ to denote the strict subset relation and $\subseteq$ when equality is also possible.

In the context of multiagent transition systems it is common to refer to the state of the system as *configuration*, so as not to confuse it with the *local states* of the agents. As standard, we use S^P to denote the set of all total functions from P to S, and if $c \in S^P$ is

a configuration over S and P, we use c_p to denote the member of c indexed by $p \in P$, i.e. $c_p = c(p)$.

▶ **Definition 2.1** (Local States, Configuration, Transaction). *Given agents $Q \subset \Pi$ and an arbitrary set S of* local states*, a* configuration *over Q and S is a member of $C := S^Q$. An* atomic transaction *over Q and S is a pair of configurations $t = c \to c' \in C^2$ such that $c \neq c'$, with $t_p := c_p \to c'_p$ for any $p \in Q$.*

An agent p is an *active participant* in t if $c_p \neq c'_p$. A transaction is *unary* if it has exactly one participant, *binary* if it has two, and *k-ary* in general.

2.2 Multiagent Transition Systems and Protocols

▶ **Definition 2.2** (Transition System). *A* transition system *is a tuple $TS = (C, c0, T)$ where C is an arbitrary set of* configurations*, $c0 \in C$ is a designated* initial configuration*, and $T \subseteq C \times C$ is a* transition relation*, with $(c, c') \in T$ also written as $c \to c' \in T$.*

A transition $c \to c' \in T$ is *enabled* from configuration c. A configuration c is *terminal* if no transitions are enabled from c. A *computation* is a (finite or infinite) sequence of configurations where for each two consecutive configurations (c, c') in the sequence, $c \to c' \in T$. A *run* is a computation starting from $c0$, which is *complete* if it is infinite or ends in a terminal configuration. A *prefix* of a computation is a finite initial segment of it.

Given agents $P \subset \Pi$, local states S with initial state $s0 \in S$, we denote configurations as $C := S^P$ and the initial configuration as $c0 := \{s0\}^P$.

A transaction and a transition are structurally identical—both are pairs of configurations—but differ in their role. A transaction is specified over its participants Q, the agents whose states are preconditions for the transaction to occur, and says nothing about agents outside Q; different transactions may have different sets of participants. A transition, by contrast, is over a fixed set of agents P, as required for the construction of a transition system. Given a set of transactions, each over its own set of participants, the closure operator (Definition 2.3) induces from them a set of transitions over a fixed P, in which non-participants remain stationary.

▶ **Definition 2.3** (Transaction Closure). *Let $P \subset \Pi$, S a set of local states, and $C := S^P$. We say p is* stationary *in a transition or transaction $t = c \to c'$ if $c_p = c'_p$. For a transaction $t = (c \to c')$ over local states S with participants Q, the* P-closure of t*, $t{\uparrow}P$, is the set of transitions over P and S defined by:*

$$t{\uparrow}P := \begin{cases} \{t' \in C^2 : \forall q \in Q.(t_q = t'_q) \wedge \forall p \in P \setminus Q.(p \text{ is stationary in } t')\} & \text{if } Q \subseteq P \\ \emptyset & \text{otherwise} \end{cases}$$

If R is a set of transactions, each $t \in R$ over some Q and S, then the P-closure of R*, $R{\uparrow}P$, is the set of transitions over P and S defined by:*

$$R{\uparrow}P := \bigcup_{t \in R} t{\uparrow}P$$

Namely, the closure over $P \supseteq Q$ of a transaction t over Q includes all transitions t' over P in which members of Q do the same in t and in t', and the rest remain in their current (arbitrary) state.

A set of transactions R over S, each with participants $Q \subseteq P$, defines a multiagent transition system over S and P as follows:

▶ **Definition 2.4** (Transactions-Based Multiagent Transition System)**.** *Given agents $P \subset \Pi$, local states S with initial local state $s0 \in S$, and a set of transactions R, each $t \in R$ over some $Q \subseteq P$ and S, the* transactions-based multiagent transition system *over P, S, and R is the transition system $TS = (S^P, \{s0\}^P, R{\uparrow}P)$.*

In other words, one can fully specify a multiagent transition system over S and P simply by providing a set of transactions over S, each with participants $Q \subseteq P$.

Given an arbitrary set of local states $\mathcal{S}$ with designated initial state $s0 \in \mathcal{S}$, a *local-states function* $S : P \mapsto 2^{\mathcal{S}}$ maps every set of agents $P \subset \Pi$ to some $S(P) \subset \mathcal{S}$ that includes $s0$ and satisfies $P \subset P' \subset \Pi \implies S(P) \subset S(P')$. The symbol S is overloaded: a set of local states in Definition 2.1, and the local-states function here, with $S(P)$ the set of local states for population P.

▶ **Definition 2.5** (Multiagent Protocol)**.** *A multiagent protocol $\mathcal{F}$ over a local-states function S is a family of multiagent transition systems that has exactly one transition system*

$$\mathcal{F}(P) = (C(P), c0(P), T(P))$$

for every $P \subset \Pi$, where $C(P) := S(P)^P$, $c0(P) := \{s0\}^P$, and $T(P)$ is its transition relation.

▶ **Definition 2.6** (Multiagent Protocol Induced by Transactions)**.** *Let S be a local-states function and R a set of transactions, each over some $Q \subset \Pi$ and $S(Q)$. A multiagent protocol $\mathcal{F}$ is* induced by *R if for each set of agents $P \subset \Pi$:*

$$\mathcal{F}(P) = (S(P)^P, \{s0\}^P, R(P){\uparrow}P)$$

where $R(P) := \{t \in R : t \text{ is over } Q \text{ and } S(Q) \text{ for some } Q \subseteq P\}$.

Hence an agent whose state a transaction reads or names is one of its participants, active or stationary.

The same transaction—such as "sync p and q"—can be carried out in many configurations, yielding different transactions that are nonetheless "the same action." An equivalence relation on transactions captures this.

▶ **Definition 2.7** (Transaction Equivalence)**.** *Given a set of transactions R, a* transaction equivalence *is an equivalence relation $\sim$ on R such that $t \sim t'$ implies t and t' have the same participants. We write $[t]$ for the equivalence class of t under $\sim$.*

An equivalence class $[t]$ is *enabled* in configuration c if some $t' \in [t]$ is enabled in c.

For example, in the specifications of Section 3, all Sync transactions over a given pair of agents—differing only in the feeds they synchronise—are equivalent: they represent "the same action" in different configurations.

▶ **Definition 2.8** (Live and Correct Run)**.** *Given a set of transactions R with equivalence $\sim$ and a designated set $\Lambda \subseteq R/\sim$ of* live *equivalence classes, a run r is* live *if for every $[t] \in \Lambda$: no suffix of r has $[t]$ enabled throughout with no member of $[t]$ taken in the suffix. A run is* correct *if it is live.*

This is a fairness condition: a live equivalence class that remains enabled is eventually taken. The set Λ distinguishes obligatory transactions—which must eventually be carried out when persistently enabled—from voluntary ones, such as posting a message, which carry no liveness obligation. Without this distinction, liveness would force every persistently-enabled Post to be executed.

2.3 Essential Agents and Global Platform Classes

The essential agents of a protocol capture its structural dependencies: which agents must be present for multi-party interaction to be possible?

▶ **Definition 2.9** (Essential Agents). *Given a multiagent protocol $\mathcal{F}$ induced by transactions R, a set of agents $E \subseteq \Pi$ is* essential *if it is a minimal set such that for every P with $P \cap E = \emptyset$, every run of $\mathcal{F}(P)$ has only unary transitions.*

The set Π always satisfies the condition, since no nonempty $P \subset \Pi$ has $P \cap \Pi = \emptyset$; and as every P is finite, the intersection of a chain of sets satisfying the condition satisfies it, so by Zorn's lemma an essential set exists and the minimum cardinality over all essential sets is well-defined. Note that a protocol may have multiple essential sets of minimum cardinality. A protocol has *all agents but one essential* if every essential set E with $\emptyset \subsetneq E \subsetneq \Pi$ has $|\Pi \setminus E| = 1$.

▶ **Definition 2.10** (Global Platform Classes). *Let $\mathcal{F}$ be a transactions-based multiagent protocol admitting an essential set E with $\emptyset \subsetneq E \subsetneq \Pi$, and let κ be the minimum cardinality of such a set. Then the* class *of $\mathcal{F}$ is defined as follows:*

1. ***Centralised**: $\kappa = 1$*
2. ***Decentralised**: $1 < \kappa < \infty$*
3. ***Federated**: $\kappa = \infty$ and some essential set E has $|\Pi \setminus E| \geq 2$*
4. ***Grassroots**: every essential set E with $\emptyset \subsetneq E \subsetneq \Pi$ has $|\Pi \setminus E| = 1$*

▶ **Observation 2.11.** *The four classes of Definition 2.10 partition the transactions-based multiagent protocols admitting an essential set E with $\emptyset \subsetneq E \subsetneq \Pi$.*

Proof. Let κ be the minimum cardinality of an essential set E with $\emptyset \subsetneq E \subsetneq \Pi$, which is well-defined. If $\kappa < \infty$, a minimum essential set has $|\Pi \setminus E| = \infty$, so the Grassroots condition fails, and exactly one of Centralised ($\kappa = 1$) and Decentralised ($1 < \kappa < \infty$) holds. If $\kappa = \infty$, neither of those holds, and exactly one of Federated (some essential set has $|\Pi \setminus E| \geq 2$) and Grassroots (every essential set has $|\Pi \setminus E| = 1$) holds, as these are complementary. Hence exactly one class applies. ◀

Namely, to classify any global platform—existing or imagined—it is enough to provide a credible multiagent atomic transactions specification of it and analyse the cardinality of the essential set of the ensuing multiagent protocol.[2]

The class of a platform depends on which agents its specification designates as servers, and that designation reflects a capability restriction: operating a fixed-address, always-on server requires a domain name and a permanently reachable host, which not every person has. The essential sets are infinite in both the federated and the grassroots class, so this restriction separates the two classes and the number of servers does not. Anyone may create a Mastodon instance, and every client depends on the instance it joined, so Mastodon is federated; were operating a server available to every agent, any two agents could serve each other and all agents but one would be essential.

In the grassroots class, $E = \Pi \setminus \{p\}$ for some p means that every agent except one is essential. The single excluded agent can only talk to oneself—any single agent alone can only

[2] It is theoretically possible for two alternative specifications to vie for being the "right" specification of some known global platform. If the two seem credible yet have different cardinalities of the minimal sets of essential agents, probably the specification with the larger set should be used for classification. We have yet to see or produce such a case.

perform unary transitions—so the essential set is as large as it can possibly be. Conversely, no two agents can be excluded: in a grassroots protocol, any two agents can interact. This essential-agents notion is the *loose* characterisation; Section 4 gives two stronger (*tight*) definitions from the literature, each of which implies it.

3 Global Social Networks: Specifications and Classification

Here we use the framework of Section 2 to provide specifications for a Facebook/X-like social network with feeds and followers for each platform class, exposing their architectural differences. For each specification we state its fairness requirement (the designation of Λ), correctness, and the cardinality of essential agents; proofs appear in Appendix A. In every specification below, Sync transactions are grouped by their participants into equivalence classes; all other transactions form singleton equivalence classes. Throughout, posts are sequences over a fixed message alphabet M and ε is the empty sequence; lowercase *posts* and "p follows q" name projections of the local state, whereas capitalised Post and Follow name transactions.

3.1 Correctness of a Social Network

The social network we specify enables agents to post messages and follow other agents to receive their posts. Regardless of architecture, any specification of such network should guarantee two fundamental properties:

▶ **Definition 3.1** (Social Network Correctness). *A* social network *is a multiagent protocol $\mathcal{F}$ induced by transactions R, equipped, for each $P \subset \Pi$, with a relation "p follows q" on configurations of $\mathcal{F}(P)$ and a projection $posts_q$ within c_p for $p, q \in P$, such that whenever p follows q in c both $posts_q$ within c_p and $posts_q$ within c_q are defined. It satisfies* Social Network Correctness *if:*

1. ***Follower Correctness**: For every $P \subset \Pi$, correct run r of $\mathcal{F}(P)$, and all $p, q \in P$, if p follows q in configuration $c \in r$, then:*
   - Safety*: $posts_q$ within c_p is a prefix of $posts_q$ within c_q.*
   - Liveness*: For any post m in $posts_q$ within c_q, eventually m appears in $posts_q$ within c_p.*
2. ***Agent Autonomy**: For all $P \subset \Pi$ and all runs of $\mathcal{F}(P)$:*
   - Post autonomy*: For any initialised agent $p \in P$ and any message m, Post(m) is enabled.*
   - Follow autonomy*: For any initialised agent $p \in P$ and $q \in P$ with $q \neq p$, if p does not follow q, then a Follow transaction is enabled.*

3.2 Centralised Platforms

Centralised platforms employ a named server agent that all other agents must interact with. We illustrate this with a social network where a central server maintains feeds for all registered users. Users can post locally but need the server to exchange content. For a post by p to reach q the following transactions have to take place: (1) p registers with the server (2) q registers with the server (3) q follows p (4) p posts (5) p syncs with the server (6) q syncs with the server. Subsequent posting by p require only the last three transactions.

▶ **Definition 3.2** (Centralised Social Network Transactions). *Given agents $P \subset \Pi$ with a named server $p \in P$ and users $P \setminus \{p\}$. Local states are sets of feeds, where a feed is a pair $(agent, posts)$ with posts a sequence. Initial local state is $\emptyset$. The transactions are $c \to c'$ where:*

- ***Register*** *over* $\{p, q\}$*:*
  - $c_q = \emptyset$ *and* $c'_q := \{(q, \varepsilon)\}$
  - $c'_p := c_p \cup \{(q, \varepsilon)\}$
- ***Post***(m) *over* $\{q\}$*:* $(q, posts) \in c_q$ *and* $c'_q := c_q \setminus \{(q, posts)\} \cup \{(q, posts \cdot m)\}$.
- ***Follow***(q') *over* $\{q, q'\}$*:* $c_q \neq \emptyset$*,* q' *has no feed in* c_q*, and* $c'_q := c_q \cup \{(q', \varepsilon)\}$
- ***Sync*** *over* $\{q, p\}$*:* q *has a feed in* c_p *and:*
  - $c'_p := c_p$ *with* q*'s posts missing in* c_p *added to* $(q, posts)$
  - $c'_q := c_q$ *with posts by followed agents missing in* c_q *added*

The transaction equivalence groups all Sync transactions over the same pair $\{q, p\}$ into one class. The live set Λ consists of these Sync classes: for any registered agent q (i.e., q has a feed in c_p), Sync over $\{q, p\}$ must eventually be taken. Post and Follow are voluntary and not in Λ.

The server p is required for any transaction to occur: Register and Sync have p as a participant, and Post and Follow require registration.

Posts reach a follower through the server: the followed agent syncs to the server and the follower syncs from it, preserving order.

▶ **Proposition 3.3.** *The centralised social network protocol satisfies Social Network Correctness.*

▶ **Proposition 3.4.** *In the centralised social network protocol,* $E = \{p\}$ *and* $|E| = 1$.

3.3 Decentralised Platforms

Bitcoin. We illustrate decentralised platforms with bootstrap agents using an abstract Bitcoin-like protocol. In particular, we do not specify preconditions for the nondeterministic AddBlock transaction. While decentralised systems typically distinguish between full nodes and lightweight nodes, we abstract away this distinction since any node can in principle become a full node. The genesis block and bootstrap node addresses are public information hardcoded in the protocol.

▶ **Definition 3.5** (Bitcoin Transactions)**.** *Given a constant set of bootstrap agents* $B \subset \Pi$ *and agents* $P \subset \Pi$*. Local states* S *are pairs (blockchain, peers) where a* block *is opaque data (carrying posts in Definition 3.6), blockchain is a sequence of blocks, and peers is a set of agents, and for configuration* c *over* S *and* P*, we let* $c_p = (chain_p, peers_p)$*. Initial local state is* (g, B) *where* g *is the genesis block. Transitions include* $c \to c'$ *where:*

- ***AddBlock***(b) *over* $\{p\}$*:* $c'_p := (chain_p \cdot b, peers_p)$.
- ***Sync*** *over* $\{p, q\}$*:* $q \in peers_p$*,* $|chain_p| < |chain_q|$ *and:*
  - $chain'_p := chain'_q := chain_q$,
  - $peers'_p := peers'_q := peers_p \cup peers_q \cup \{p\}$

If the chain of p is not a prefix of the chain of q upon a Sync over $\{p, q\}$, then the blocks in p that are inconsistent with the chain in q are *abandoned* and we say that the chain of p has undergone *reorg*. The correctness of the protocol depends on the probability of an AddBlock transaction being much lower than the probability of a Sync transaction [24, 43]. This in turn implies that the probability of a reorg abandoning a block diminishes quickly as a function of how deep the block is "buried", namely how many blocks follow it in the blockchain [40, 27]. Moreover, the probabilistic argument underscores the importance of the bootstrap nodes: It depends on them forming a connected component and on all active

agents joining this component; the argument falls apart if multiple connected components form and grow independently [28, 5].

Decentralised Social Network. Each agent maintains their own posts and publishes them directly to the blockchain. Chain reorganizations require rolling back the publication pointer for posts in abandoned blocks.

For a post by p to reach q the following unbounded sequence of transactions have to take place: (1) p posts block b (2) a sequence of Sync transactions in which b is part of the longer chain, the last of which is with q. If a Sync transaction with p entails a reorg in which the block b is abandoned , it has to be posted again.

▶ **Definition 3.6** (Decentralised Social Network Transactions). *Given agents $P \subset \Pi$ with bootstrap nodes $B \subset P$. Local state is $(blockchain, peers, posts, pointer)$ where blockchain is a sequence of blocks, peers is a set of agents, posts is a sequence of posts, and pointer indexes published posts, and for configuration c over S and P, we let $c_p = (chain_p, peers_p, posts_p, pointer_p)$. Initial local state is $(g, B, \varepsilon, 0)$. The transactions are $c \to c'$ where:*

- ***Post***(m) *over* $\{p\}$*:* $c'_p := (chain_p, peers_p, posts_p \cdot m, pointer_p)$.
- ***AddBlock***(b) *over* $\{p\}$*:* b *contains posts from* $pointer_p$ *to* $|posts_p|$*, then* $c'_p := (chain_p \cdot b, peers_p, posts_p, |posts_p|)$.
- ***Sync*** *over* $\{p, q\}$*:* $q \in peers_p$*,* $|chain_p| < |chain_q|$ *and:*
  - $chain'_p := chain'_q := chain_q$
  - $peers'_p := peers'_q := peers_p \cup peers_q \cup \{p\}$
  - $pointer'_q := pointer_q$*, and* $pointer'_p$ *is the number of posts by* p *contained in the blocks of* x*, where* x *is the longest common prefix of* $chain_p$ *and* $chain_q$

While several proposals exist for blockchain-based social networks [45, 12, 15, 16], our specification exposes fundamental architectural mismatches:

- **Replication**: All posts must be replicated to all agents and stored by all agents, regardless of who follows whom
- **Consensus**: Every post requires global consensus.
- **Instability**: Posts may be reordered after publication and responses may appear before their referents.

Blockchain efficiency can be improved by sharding [30, 61], layering [44, 19], and side channels [21, 39], but the issues raised above cannot be addressed short of operating the social network via a different architecture. Perhaps the key reason for that is that social networks do not need consensus to operate (see the grassroots social network protocol below, Definition 3.12), so there is no reason to pay the price for consensus in order to realise them.

The live set Λ consists of the Sync classes: for any agents p, q where $q \in peers_p$, Sync over $\{p, q\}$ must eventually be taken. Post and AddBlock are not in Λ.

The bootstrap nodes B are hardcoded in the initial state as the peer set of every agent; without them, no Sync is possible.

Every agent replicates the whole blockchain, so all posts reach everyone and follower correctness holds trivially.

▶ **Proposition 3.7.** *The decentralised social network protocol satisfies Social Network Correctness.*

▶ **Proposition 3.8.** *In the decentralised social network protocol, $E = B$ and $1 < |E| < \infty$.*

3.4 Federated Platforms

Federated Social Network. Multiple servers cooperate through federation. Users bind to home servers with qualified identities (e.g., p@s). Federation occurs when users follow remote users.

For a post by p to reach q the following eight transactions have to take place: (1) p registers with server r (2) q registers with server s (3) $q@s$ follows $p@r$ (4) $q@s$ syncs with the server s. (5) $p@r$ posts (6) $p@r$ syncs with the server r (7) server r syncs with server s (8) $q@s$ syncs with server s. Subsequent posts by p require only the last four transactions.

▶ **Definition 3.9** (Federated Social Network Transactions). *Given agents $P \subset \Pi$ with designated servers $Q \subset P$ and clients $P \setminus Q$. Local states are sets of feeds, where a feed is $(agent@server, posts)$ with posts a sequence. Initial local state is $\emptyset$. Transactions are $c \to c'$ where:*

- ***Register*** *over $\{p, s\}$: $c_p = \emptyset$, $c'_p := \{(p@s, \varepsilon)\}$, and $c'_s := c_s \cup \{(p@s, \varepsilon)\}$*
- ***Post***(m) *over $\{p\}$: $(p@s, posts) \in c_p$ and $c'_p := c_p \setminus \{(p@s, posts)\} \cup \{(p@s, posts \cdot m)\}$.*
- ***Follow***$(q@s')$ *over $\{p, q, s'\}$: $c_p \neq \emptyset$, $q@s'$ has no feed in c_p, and $c'_p := c_p \cup \{(q@s', \varepsilon)\}$*
- ***Sync*** *over $\{a, b\}$:*
  - *If $a \in P \setminus Q$ and $b \in Q$ where $a@b$ has a feed in c_b:*
    - *$c'_b := c_b$ with $(a@b, posts)$ updated to include any posts from c_a*
    - *$c'_a := c_a$ with feeds of followed users updated from c_b's copies*
  - *If $a, b \in Q$ and $a \neq b$: For each $p@a$ with a feed in c_b, or $q@b$ with a feed in c_a:*
    - *If $(p@a, posts) \in c_a$ and $(p@a, posts') \in c_b$: update to longer sequence in both*
    - *If $(q@b, posts) \in c_b$ and $(q@b, posts') \in c_a$: update to longer sequence in both*

The live set Λ consists of two kinds of Sync classes: (i) client-server: for any registered client p at server s (i.e., $p@s$ has a feed in c_s), Sync over $\{p, s\}$ must eventually be taken; (ii) server-server: for any servers s_1, s_2 where $p@s_1$ has a feed in c_{s_2} for some agent p, Sync over $\{s_1, s_2\}$ must eventually be taken.

Servers are required for any transaction to occur: Register requires a server, client-server and server-server Sync both require at least one server as participant, and Post and Follow require registration. Without any server, no client can initialise.

The designation of Q is the capability restriction of Section 2.3: operating a fixed-address, always-on server is not available to every agent. Were it available to every agent, the same transactions with $Q = P$ would specify a grassroots platform, with all agents but one essential.

Posts propagate from a client to its server, between servers, and to the following client's server, preserving order along the way.

▶ **Proposition 3.10.** *The federated social network protocol satisfies Social Network Correctness.*

▶ **Proposition 3.11.** *In the federated social network protocol, $E = Q$, $|E| = \infty$, and $|\Pi \setminus E| \geq 2$.*

3.5 Grassroots Platforms

Grassroots Social Network. For a post by p to reach q the following five transactions have to take place: (1) p initialises (2) q initialises (3) q follows p (4) p posts (5) p syncs with q. Subsequent posting by p require only the last two transactions.

▶ **Definition 3.12** (Grassroots Social Network Transactions). *Given agents $P \subset \Pi$. Local states are sets of feeds $(agent, posts)$, where posts is a sequence. Initial local state is $\emptyset$. Transactions are $c \to c'$ where:*

- ***Initialise*** *over $\{p\}$: $c_p = \emptyset$ and $c'_p := \{(p, \varepsilon)\}$*
- ***Post***(m) *over $\{p\}$: $(p, s) \in c_p$ and $c'_p := c_p \setminus \{(p, s)\} \cup \{(p, s \cdot m)\}$*
- ***Follow***(q) *over $\{p, q\}$: $p \neq q$, q has no feed in c_p, $c'_p := c_p \cup \{(q, \varepsilon)\}$*
- ***Sync*** *over $\{p, q\}$: for any r with $(r, s) \in c_p$ and $(r, s') \in c_q$:*
  - *If $|s| < |s'|$: update (r, s) to (r, s') in c'_p*
  - *If $|s'| < |s|$: update (r, s') to (r, s) in c'_q*

The live set Λ consists of the Sync classes: for any agents p, q where q has a feed in c_p, Sync over $\{p, q\}$ must eventually be taken. Initialise, Post, and Follow are voluntary and not in Λ.

A follower obtains posts by direct Sync with the followed agent, preserving order, with no intermediary.

▶ **Proposition 3.13.** *The grassroots social network protocol satisfies Social Network Correctness.*

▶ **Proposition 3.14.** *In the grassroots social network protocol, $E = \Pi \setminus \{p\}$ for some $p \in \Pi$.*

4 Grassroots Platforms: Two Characterisations

Grassroots platforms have two previous definitions [53, 57, 32]. Here we present both, prove them incomparable, prove that each implies all agents but one are essential, placing grassroots platforms in the fourth class of Definition 2.10, and prove that both implications are strict.

4.1 The Original Definition

The original definition of grassroots protocols [53, 57] is based on the notion of interactivity.

▶ **Definition 4.1** (Projection). *For a configuration $c \in S^{P'}$ and subset $P \subseteq P'$, the* projection *of c to P, denoted c/P, is the configuration in S^P defined by $(c/P)_p = c_p$ for all $p \in P$.*

▶ **Definition 4.2** (Computation Notation). *For configurations c, c' in a transition system, we write $c \xrightarrow{*} c'$ if there exists a finite computation (sequence of transitions) from c to c'. Specifically, there exist configurations $c = c_0, c_1, \ldots, c_n = c'$ where $c_i \to c_{i+1}$ is a transition for each $i < n$.*

▶ **Definition 4.3** (Interactive Protocol). *A protocol $\mathcal{F}$ is* interactive *if for every $\emptyset \subset P \subset P' \subseteq \Pi$ and every configuration $c \in C(P')$ such that $c/P \in C(P)$, there is a computation $c \xrightarrow{*} c'$ of $\mathcal{F}(P')$ for which $c'/P \notin C(P)$.*

The interactive property requires that agents in P can always potentially interact with agents in $P' \setminus P$, leaving "alien traces" in their local states that could not have been produced by P operating alone.

▶ **Definition 4.4** (Grassroots Protocol (Original)). *An atomic transactions-based protocol $\mathcal{F}$ is* grassroots *if it is interactive.*

We note that the original definition of grassroots protocols [53] included a notion of obliviousness, however subsequent work [57] showed that any transactions-based multiagent protocol is oblivious. Hence we bypass it here.

▶ **Theorem 4.5.** *In a grassroots protocol (under the original definition) all agents but one are essential.*

Proof. Assume $\mathcal{F}$ is grassroots, then by definition it is interactive. Suppose for contradiction that some essential set E with $|\Pi \setminus E| \geq 2$, so there exist at least two agents $p \neq q$ with $p, q \notin E$.

Let $P = \{p\}$ and $P' = \{p, q\}$. Note that $P \subset P'$ and $P' \cap E = \emptyset$.

Since E is essential and $P' \cap E = \emptyset$, every run of $\mathcal{F}(P')$ has only unary transitions. A unary transition of $\mathcal{F}(P')$ active at p is a transition of $\mathcal{F}(P)$, as its transaction is over $\{p\}$ and $S(\{p\})$ and so is in $R(P)$; a transition active at q leaves p's state unchanged. Hence for every run $c0(P') \xrightarrow{*} c'$ of $\mathcal{F}(P')$, the trace of p's state is reproducible by a run of $\mathcal{F}(P)$, so $c'/P \in C(P)$.

But $\mathcal{F}$ is interactive, so for $P = \{p\} \subset P' = \{p, q\}$ and $c = c0(P')$, which has $c/P = c0(P) \in C(P)$, there is a computation $c \xrightarrow{*} c'$ of $\mathcal{F}(P')$ with $c'/P \notin C(P)$; it is a run of $\mathcal{F}(P')$, contradicting the previous paragraph. ◀

4.2 The Interleaving-Based Definition

A recent alternative definition [32] captures the informal notion of grassroots directly: two disjoint groups of agents can each operate independently—their interleaved correct runs are correct runs of the combined system—yet the combined system offers genuinely new behaviours that neither group could produce on its own. To distinguish the three interleaving-based notions from their original counterparts, we mark them with a prime: oblivious′, interactive′, grassroots′.

▶ **Definition 4.6** (Interleaving)**.** *Let $P, P' \subset \Pi$ be disjoint nonempty sets of agents, $r = c_0, c_1, \ldots$ a run of $\mathcal{F}(P)$, and $r' = d_0, d_1, \ldots$ a run of $\mathcal{F}(P')$. An* interleaving *of r and r' is a sequence $e_0, e_1, \ldots$ of configurations in $C(P \cup P')$ for which there exist non-decreasing sequences of indices $(i_k)_{k\geq 0}$ and $(j_k)_{k\geq 0}$ with $i_0 = j_0 = 0$ such that for every $k \geq 0$:*

1. *$(e_k)_p = (c_{i_k})_p$ for every $p \in P$,*
2. *$(e_k)_q = (d_{j_k})_q$ for every $q \in P'$,*
3. *if e_{k+1} exists, then exactly one of: (i) $i_{k+1} = i_k + 1$ and $j_{k+1} = j_k$ (a P-step), or (ii) $i_{k+1} = i_k$ and $j_{k+1} = j_k + 1$ (a P'-step).*

Moreover, if r is finite of length n then $i_k = n$ for some k, and if r is infinite then for every $m \geq 0$ there is a k with $i_k = m$; likewise for r' and (j_k).

▶ **Definition 4.7** (Interaction)**.** *Let $Q \subset \Pi$. A transition $e \to e'$ of $\mathcal{F}(Q)$ is an* interaction between $x \neq y \in Q$ *if $e/\{x\} \neq e'/\{x\}$ and $e/\{y\} \neq e'/\{y\}$, and $e/\{x\} \to e'/\{x\}$ is not a transition of $\mathcal{F}(\{x\})$ or $e/\{y\} \to e'/\{y\}$ is not a transition of $\mathcal{F}(\{y\})$. It is an* interaction between *disjoint nonempty $P, P' \subseteq Q$ if it is an interaction between some $x \in P$ and some $y \in P'$.*

▶ **Definition 4.8** (Oblivious′, Interactive′, Grassroots′)**.** *A protocol $\mathcal{F}$ is:*

1. oblivious′ *if for every disjoint nonempty $P, P' \subset \Pi$, every interleaving of a correct run of $\mathcal{F}(P)$ and a correct run of $\mathcal{F}(P')$ is a correct run of $\mathcal{F}(P \cup P')$.*
2. interactive′ *if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P' is a prefix of a correct run of $\mathcal{F}(Q)$ that contains one.*
3. grassroots′ *if it is oblivious′ and interactive′.*

Being oblivious′ means that two disjoint groups of agents, each running the protocol independently and correctly, do not interfere with each other. Being interactive′ means that two groups that have not yet interacted can always still do so: whatever they have done apart, the combined system has a correct continuation in which a single transition engages agents of both, which no interleaving of their separate runs can produce.

▶ **Theorem 4.9.** *In an interactive′ protocol all agents but one are essential.*

Proof. Suppose for contradiction that some essential set E has $|\Pi \setminus E| \geq 2$. Then there exist at least two agents $p \neq q$ with $p, q \notin E$. Let $Q = \{p, q\}$, $P = \{p\}$ and $P' = \{q\}$. Since $Q \cap E = \emptyset$ and E is essential, every run of $\mathcal{F}(Q)$ has only unary transitions, so no transition of such a run changes the local states of both p and q, and by Definition 4.7 none is an interaction between P and P'. Hence the empty prefix of a correct run of $\mathcal{F}(Q)$ contains no interaction between P and P' and is a prefix of no correct run that contains one, contradicting the interactivity′ of $\mathcal{F}$. ◀

Since grassroots′ protocols are interactive′, the same holds for them.

Relation between the two definitions. The two definitions of grassroots are incomparable in general—neither implies the other—but for all platforms studied thus far, both definitions agree. The interleaving-based definition [32] is simpler and better captures the informal notion of grassroots: multiple instances can form and operate independently, yet may coalesce when interconnected.

Two protocols separate the definitions, one in each direction.

▶ **Definition 4.10** (Follow Only)**.** *Given agents $P \subset \Pi$. A local state is a set $F \subseteq P$ of the agents followed; the initial local state is $\emptyset$. The one transaction is:*
- ***Follow***(q) *over $\{p, q\}$: $p \neq q$ and $q \notin F_p$; $F'_p := F_p \cup \{q\}$*

The live set Λ is empty.

▶ **Definition 4.11** (Paired Counter)**.** *Given agents $P \subset \Pi$. A local state is a pair (n, a) with $n \in \mathbb{N}$ and $a \in P \cup \{\perp\}$; the initial local state is $(0, \perp)$. The one transaction is:*
- ***Tick*** *over $\{p, q\}$: $n'_p := n_p + 1$ and $n'_q := n_q + 1$; a_p and a_q unchanged*

The live set Λ is empty. No transaction changes the second component of a local state; it is there so that the local-states function is strictly monotone.

▶ **Proposition 4.12.** *Follow Only is interactive and not interactive′, and Paired Counter is interactive′ and not interactive. Hence neither definition implies the other.*

Both definitions are *tight*: each implies the *loose* characterisation of Definition 2.10—all agents but one essential—but, as discussed below, not conversely. The question arises, which platforms qualify as grassroots under the loose characterisation but under neither tight definition? As far-fetched as it may sound, we believe these are platforms in which participants sacrifice their dignity and forgo their basic human rights. In one platform participants forgo their freedom of speech, and runs have a suffix in which no two agents communicate. In another, after a finite prefix a leader is chosen who cannot be deposed and who thereafter mediates all agent-to-agent communication. We specify both and prove that each has all agents but one essential, but is grassroots under neither tight definition. The mathematical and practical implications of the difference between the tight and loose grassroots notions remains to be studied.

▶ **Definition 4.13** (Silenced Speech). *Given agents $P \subset \Pi$. A local state is a pair (v, M) with $v \in \{open, silent\}$ and $M \subseteq P$ the agents spoken with; the initial local state is $(open, \emptyset)$. Transactions are:*
- ***Speak*** *over $\{p, q\}$: $v_p = v_q = open$ and $q \notin M_p$; $M'_p := M_p \cup \{q\}$ and $M'_q := M_q \cup \{p\}$*
- ***Silence*** *over $\{p\}$: $v_p = open$; $v'_p := silent$*

The live set Λ is empty: both transactions are voluntary.

▶ **Definition 4.14** (Entrenched Leader). *Given agents $P \subset \Pi$. A local state is a pair (ℓ, M) with $\ell \in P \cup \{\perp\}$ the agent's leader and $M \subseteq P$ the agents communicated with; the initial local state is $(\perp, \emptyset)$. Transactions are:*
- ***Meet*** *over $\{p, q\}$: $\ell_p = \ell_q = \perp$ and $q \notin M_p$; $M'_p := M_p \cup \{q\}$ and $M'_q := M_q \cup \{p\}$*
- ***Elect***(r) *over $\{p, r\}$: $\ell_p = \perp$; $\ell'_p := r$*
- ***Relay*** *over $\{p, q\}$: $\ell_p = q$ and $q \notin M_p$; $M'_p := M_p \cup \{q\}$ and $M'_q := M_q \cup \{p\}$*

The live set Λ is empty.

Agents communicate freely until they elect, and an election cannot be undone; thereafter an agent communicates only with its leader.

▶ **Proposition 4.15.** *In each of Silenced Speech and Entrenched Leader all agents but one are essential, and neither is interactive nor interactive′.*

We can now show that the grassroots social network protocol (Definition 3.12) satisfies both definitions.

▶ **Proposition 4.16.** *The grassroots social network protocol is grassroots under the original definition and grassroots′ under the interleaving-based definition.*

5 Related Work

The characterisation of distributed systems has been a fundamental challenge in computer science since the field's inception. Our work builds upon and extends several research threads: formal models of distributed computation, atomic transactions in distributed systems, peer-to-peer architectures, and the emerging theory of grassroots computing.

5.1 Formal Models of Distributed Systems

Lynch's comprehensive treatment of distributed algorithms [35] provides foundational definitions for distributed systems, though primarily focused on consensus and synchronization rather than architectural classification. Tel's work [60] similarly emphasizes algorithmic aspects without addressing the structural characterisation we identify. Attiya and Welch [6] present a computational model based on message passing and shared memory, but do not address the architectural classification of platforms.

The closest work to ours in spirit is Angluin's seminal paper on local and global properties in networks of processors [4], which distinguishes between problems solvable with purely local information versus those requiring global coordination. However, Angluin focuses on computational complexity rather than architectural characterisation, and does not address the spectrum from centralized to grassroots that we formalize.

The notion of essential agents is analogous to other threshold concepts in distributed computing, such as the FLP impossibility result requiring failure detectors for consensus, or Byzantine agreement requiring $n > 3f$ participants.

5.2 Atomic Transactions and Distributed Computing

Atomic transactions have been extensively studied in distributed database systems [8, 26, 63]. The integration of atomic transactions into distributed computing models was pioneered by Lynch and colleagues [36], though their focus was on correctness properties rather than using transactions as a basis for system classification.

More recent work has explored atomic transactions in the context of blockchain systems [29], though this literature assumes a specific architectural model (blockchain) rather than using transactions to characterize different architectures. The extension of process calculi with atomic transactions [1, 18] provides formal semantics but has not been applied to classify system architectures.

5.3 Peer-to-Peer and Decentralized Systems

The peer-to-peer systems literature provides extensive practical examples but limited formal characterisation. Lua et al. [33] survey P2P overlay networks, categorizing them by topology (structured vs. unstructured) rather than by essential agent requirements. Androutsellis-Theotokis and Spinellis [3] classify P2P systems by application domain without addressing the fundamental architectural distinctions we identify.

Scuttlebutt [59] appears to be grassroots, though it has not been formally proven to be so. BitTorrent [17], the Dat protocol [41] and IPFS [7] occupy an intermediate position, requiring bootstrap nodes similar to our characterisation of decentralized platforms.

6 Conclusion

We introduced essential agents—minimal sets whose removal prevents all non-unary transitions—and proved their cardinality partitions global platforms into four classes: centralised (1), decentralised (finite >1), federated (infinite, not universal), and grassroots (universal). We proved that the two previous definitions of grassroots platforms are incomparable, that each implies all agents but one are essential, and that both implications are strict. This work provides the first mathematical framework for studying today's most important family of computer systems.

Use of AI tools. An AI assistant was used throughout the preparation of this paper: for discussion of the definitions and the proofs, for drafting and revising prose, and for checking citations. The authors are accountable for all of its content.

References

1 Lucia Acciai, Michele Boreale, and Silvano Dal Zilio. A concurrent calculus with atomic transactions. In *European Symposium on Programming*, pages 48–63. Springer, 2007.

2 Algorand Foundation. Algorand goes peer-to-peer (P2P): A new era for the network, 12 2025. URL: `https://algorand.co/blog/algorand-goes-peer-to-peer-a-new-era-for-the-network`.

3 Stephanos Androutsellis-Theotokis and Diomidis Spinellis. A survey of peer-to-peer content distribution technologies. *ACM Computing Surveys*, 36(4):335–371, 2004.

4 Dana Angluin. Local and global properties in networks of processors. *Proceedings of the 12th Annual ACM Symposium on Theory of Computing*, pages 82–93, 1980.

5 Maria Apostolaki, Aviv Zohar, and Laurent Vanbever. Hijacking bitcoin: Routing attacks on cryptocurrencies. In *IEEE Symposium on Security and Privacy*, pages 375–392, 2017.

6 Hagit Attiya and Jennifer Welch. *Distributed computing: fundamentals, simulations, and advanced topics*, volume 19. John Wiley & Sons, 2004.

7 Juan Benet. Ipfs-content addressed, versioned, p2p file system. *arXiv preprint arXiv:1407.3561*, 2014.

8 Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

9 Bittensor Foundation. Bittensor: A decentralized machine learning protocol. `https://bittensor.com`, 2024.

10 Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative inference and fine-tuning of large models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, pages 558–568. Association for Computational Linguistics, 2023.

11 Briar Project. Briar: Secure messaging, anywhere. `https://briarproject.org`, 2018.

12 Sonja Buchegger, Doris Schiöberg, Le-Hung Vu, and Anwitaman Datta. Peerson: P2p social networking: early experiences and insights. In *Proceedings of the Second ACM EuroSys Workshop on Social Network Systems*, pages 46–52, 2009. `doi:10.1145/1578002.1578010`.

13 Vitalik Buterin. A next-generation smart contract and decentralized application platform. *white paper*, 3(37), 2014. URL: `https://ethereum.org/whitepaper/`.

14 Vitalik Buterin. Governance, part 2: Plutocracy is still bad, 2018. Available at `https://vitalik.eth.limo/general/2018/03/28/plutocracy.html`. URL: `https://vitalik.eth.limo/general/2018/03/28/plutocracy.html`.

15 Gregory Chockler, Roie Melamed, Yoav Tock, and Roman Vitenberg. Constructing scalable overlays for pub-sub with many topics. In *Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing*, pages 109–118, 2007.

16 Gregory Chockler, Roie Melamed, Yoav Tock, and Roman Vitenberg. Spidercast: a scalable interest-aware overlay for topic-based pub/sub communication. In *Proceedings of the 2007 inaugural international conference on Distributed event-based systems*, pages 14–25, 2007.

17 Bram Cohen. Incentives build robustness in bittorrent. In *Workshop on Economics of Peer-to-Peer Systems*, volume 6, pages 68–72, Berkeley, CA, USA, June 2003.

18 Edsko de Vries, Vasileios Koutavas, and Matthew Hennessy. Communicating transactions. In *International Conference on Concurrency Theory*, pages 569–583. Springer, 2010.

19 Christian Decker and Roger Wattenhofer. A fast and scalable payment network with bitcoin duplex micropayment channels. In *Symposium on Self-Stabilizing Systems*, pages 3–18. Springer, 2015.

20 Discord Inc. Discord - chat for communities and friends, 2015. Voice, video, and text communication platform. URL: `https://discord.com`.

21 Stefan Dziembowski, Lisa Eckey, Sebastian Faust, and Daniel Malinowski. General state channel networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 949–966, 2018.

22 Ian Foster, Carl Kesselman, and Steven Tuecke. The anatomy of the grid: Enabling scalable virtual organizations. *International Journal of High Performance Computing Applications*, 15(3):200–222, 2001.

23 Framasoft. Peertube: A decentralized video hosting network. `https://joinpeertube.org`, 2018.

24 Juan Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol: Analysis and applications. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 281–310. Springer, 2015.

25 Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles*, pages 51–68, 2017.

26 Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

27 Dongning Guo and Ling Ren. Bitcoin's latency–security analysis made simple. In *ACM Conference on Advances in Financial Technologies*, pages 100–113, 2022.

28 Ethan Heilman, Alison Kendler, Aviv Zohar, and Sharon Goldberg. Eclipse attacks on bitcoin's peer-to-peer network. In *24th USENIX Security Symposium*, pages 129–144, 2015.

29 Maurice Herlihy. Atomic cross-chain swaps. In *Proceedings of the 2018 ACM symposium on principles of distributed computing*, pages 245–254, 2018.

30 Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. Omniledger: A secure, scale-out decentralized ledger via sharding. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 583–598. IEEE, 2018.

31 Andrew Lewis-Pye, Oded Naor, and Ehud Shapiro. Grassroots flash: A payment system for grassroots cryptocurrencies. *arXiv preprint arXiv:2309.13191*, 2023. URL: `https://arxiv.org/abs/2309.13191`.

32 Andy Lewis-Pye and Ehud Shapiro. Volition-guarded multiagent atomic transactions: Describing people and their machines. *arXiv preprint arXiv:2604.25596*, 2026. URL: `https://arxiv.org/abs/2604.25596`.

33 Eng Keong Lua, Jon Crowcroft, Marcelo Pias, Ravi Sharma, and Steven Lim. A survey and comparison of peer-to-peer overlay network schemes. *IEEE Communications Surveys & Tutorials*, 7(2):72–93, 2005.

34 Joshua Lund. Technology preview: Sealed sender for signal. Signal Blog, October 2018. URL: `https://signal.org/blog/sealed-sender/`.

35 Nancy A. Lynch. *Distributed Algorithms*. Morgan Kaufmann, 1996.

36 Nancy A. Lynch, Michael Merritt, William Weihl, and Alan Fekete. Atomic transactions. *Morgan Kaufmann*, 1988.

37 Matrix.org Foundation. Matrix specification. Technical specification, The Matrix.org Foundation, 2019. Decentralized communication protocol specification. URL: `https://spec.matrix.org`.

38 Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the xor metric. In *Peer-to-Peer Systems: First International Workshop, IPTPS 2002*, pages 53–65, Cambridge, MA, USA, March 2002. Springer. `doi:10.1007/3-540-45748-8\_5`.

39 Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry. Sprites and state channels: Payment networks that go faster than lightning. In *International Conference on Financial Cryptography and Data Security*, pages 508–526. Springer, 2017.

40 Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008. URL: `https://bitcoin.org/bitcoin.pdf`.

41 Maxwell Ogden, Karissa McKelvey, and Mathias Buus. Dat - distributed dataset synchronization and versioning. `https://github.com/datprotocol/docs`, 2017. White paper, version 2017-01-17.

42 Openbazaar: A peer-to-peer marketplace. `https://openbazaar.org`, 2014.

43 Rafael Pass, Lior Seeman, and Abhi Shelat. Analysis of the blockchain protocol in asynchronous networks. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 643–673. Springer, 2017.

44 Joseph Poon and Thaddeus Dryja. The bitcoin lightning network: Scalable off-chain instant payments. *Draft version 0.5.9.2*, 2016. URL: `https://lightning.network/lightning-network-paper.pdf`.

45 Project Liberty. Decentralized social networking protocol (dsnp) specification. `https://dsnp.org/`, 2021. Version 1.0.

46 Aravindh Raman, Sagar Joglekar, Emiliano De Cristofaro, Nishanth Sastry, and Gareth Tyson. Challenges in the decentralised web: The mastodon case. In *Proceedings of the internet measurement conference*, pages 217–229, 2019. `doi:10.1145/3355369.3355572`.

47 Eugen Rochko. Mastodon: A decentralized social network based on open web protocols. `https://joinmastodon.org`, 2016–. Open-source ActivityPub server implementation. URL: `https://joinmastodon.org/`.

48 Peter Saint-Andre. RFC 3920: Extensible Messaging and Presence Protocol (XMPP): Core. Technical report, IETF, 2004.

49 Trebor Scholz. *Platform Cooperativism: Challenging the Corporate Sharing Economy*. Rosa Luxemburg Stiftung, New York, 2016.

50 David Schwartz, Noah Youngs, and Arthur Britto. The ripple protocol consensus algorithm. *Ripple Labs White Paper*, 2014.

51 Ehud Shapiro. Multiagent transition systems: Protocol-stack mathematics for distributed computing. *arXiv preprint arXiv:2112.13650*, 2021. URL: `https://arxiv.org/abs/2112.13650`.

52 Ehud Shapiro. Grassroots currencies: A foundation for a grassroots digital economy. *arXiv preprint arXiv:2202.05619*, 2022. URL: `https://arxiv.org/abs/2202.05619`.

53 Ehud Shapiro. Grassroots distributed systems: Concept, examples, implementation and applications (brief announcement). In *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391)*, pages 47:1, 47:7, Italy, 2023. LIPICS. URL: `https://arxiv.org/abs/2301.04391`.

54 Ehud Shapiro. Grassroots social networking: Serverless, permissionless protocols for twitter/linkedin/whatsapp. In *OASIS '23*. Association for Computing Machinery, 2023. `doi:10.1145/3599696.3612898`.

55 Ehud Shapiro. Glp: A grassroots, multiagent, concurrent, logic programming language for ai (full version). *arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133*, 2025. URL: `https://arxiv.org/abs/2510.15747`.

56 Ehud Shapiro. Grassroots bonds as a foundation for market liquidity. arXiv:2603.13671, 2026. URL: `https://arxiv.org/abs/2603.13671`, `arXiv:2603.13671`.

57 Ehud Shapiro. Grassroots platforms with atomic transactions: Social graphs, cryptocurrencies, and democratic federations. In *Proceedings of the 27th International Conference on Distributed Computing and Networking*, pages 71–81, 2026. arXiv preprint arXiv:2502.11299. `doi:10.1145/3772290.3772309`.

58 Nimrod Talmon and Ehud Shapiro. Grassroots federation: Fair democratic governance at scale. *arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26*, 2025. URL: `https://arxiv.org/abs/2505.02208`.

59 Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In *Proceedings of the 6th ACM conference on information-centric networking*, pages 1–11, 2019. `doi:10.1145/3357150.3357396`.

60 Gerard Tel. *Introduction to Distributed Algorithms*. Cambridge University Press, 2nd edition, 2000.

61 Jiaping Wang and Hao Wang. Monoxide: Scale out blockchains with asynchronous consensus zones. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 95–112, 2019.

62 Shuai Wang, Wenwen Ding, Juanjuan Li, Yong Yuan, Liwei Ouyang, and Fei-Yue Wang. Decentralized autonomous organizations: Concept, model, and applications. *IEEE Transactions on Computational Social Systems*, 6(5):870–878, 2019. `doi:10.1109/tcss.2019.2938190`.

63 Gerhard Weikum and Gottfried Vossen. *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. Morgan Kaufmann, 2001.

64 Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger, 2014.

65 Shoshana Zuboff. *The age of surveillance capitalism: The fight for a human future at the new frontier of power*. Public Affairs, US, 2019. URL: `https://www.hachettebookgroup.com/titles/shoshana-zuboff/the-age-of-surveillance-capitalism/9781610395694/`.

66 Shoshana Zuboff. Surveillance capitalism or democracy? the death match of institutional orders and the politics of knowledge in our information civilization. *Organization Theory*, 3(3):26317877221129290, 2022. `doi:10.1177/26317877221129290`.

A Proofs

▶ **Proposition 3.3.** *The centralised social network protocol satisfies Social Network Correctness.*

Proof. **Follower Correctness.** *Safety*: If $(q, posts) \in c_p$, then either (i) p has not synced with server s since following q, in which case $posts = \varepsilon$, or (ii) p has synced, receiving q's posts from c_s. Since the server only appends to $(q, posts)$ in c_s during Sync with q, and p receives these during Sync with s, $posts$ in c_p is always a prefix of $posts$ in c_q. *Liveness*: If p follows q and q has posted m, then by liveness of the Sync class for $\{q, s\}$, Sync over $\{q, s\}$ eventually occurs, adding m to $(q, posts)$ in c_s. Subsequently, Sync over $\{p, s\}$ occurs, copying the updated posts to c_p.

Agent Autonomy. Post(m) over $\{q\}$ requires only $(q, posts) \in c_q$, which holds after registration. Follow(q') over $\{q\}$ requires $c_q \neq \emptyset$ and q' has no feed in c_q, always enabled for unfollowed agents. ◀

▶ **Proposition 3.4.** *In the centralised social network protocol, $E = \{p\}$ and $|E| = 1$.*

Proof. Without the server ($p \notin P$), Register and Sync are unavailable, as both have p as a participant; so no agent registers, and Post and Follow, which require a registered agent, are never enabled. Hence $\{p\}$ is essential. Minimality: the empty set is not essential since Sync over $\{q, p\}$ is a non-unary transition enabled when $p \in P$. Therefore $E = \{p\}$ and $|E| = 1$. ◀

▶ **Proposition 3.7.** *The decentralised social network protocol satisfies Social Network Correctness.*

Proof. **Follower Correctness.** This holds vacuously as there is no Follow transaction. All agents receive all posts via blockchain replication.

Agent Autonomy. Post(m) over $\{p\}$ is always enabled as it only appends to $posts_p$. Follow autonomy is not applicable (no Follow transaction defined). ◀

▶ **Proposition 3.8.** *In the decentralised social network protocol, $E = B$ and $1 < |E| < \infty$.*

Proof. The initial local state is (g, B), so $peers_p = B$ for every agent p. If $P \cap B = \emptyset$, then for every $p \in P$ and every $q \in peers_p = B$, $q \notin P$, so Sync over $\{p, q\}$ is never enabled. Only AddBlock and Post (both unary) remain. Hence B is essential.

Minimality: for any $b \in B$, consider $P = \{q, b\}$ where $q \notin B$. Since $b \in peers_q = B$, Sync over $\{q, b\}$ is enabled—a non-unary transition. Hence $B \setminus \{b\}$ does not suffice, and $E = B$ is minimal. No single agent is essential: for any a, taking a client q and a bootstrap node $b \in B$ with $q, b \neq a$ (possible as $|B| > 1$), Sync over $\{q, b\}$ is enabled while $a \notin \{q, b\}$. Hence $\kappa > 1$, and since B is essential and finite, $1 < \kappa < \infty$—the protocol is decentralised. ◀

▶ **Proposition 3.10.** *The federated social network protocol satisfies Social Network Correctness.*

Proof. **Follower Correctness.** *Safety*: If $(q@s', posts) \in c_p$, then $posts$ was obtained via Sync with p's home server, which obtained it via federation from s' (if $s' \neq s$) or directly (if $s' = s$). Server-to-server sync preserves prefix ordering, so $posts$ is a prefix of q's actual posts. *Liveness*: If p follows $q@s'$ and q posts m: (i) by liveness of the client-server Sync class, Sync over $\{q, s'\}$ occurs, updating $(q@s', posts)$ in $c_{s'}$; (ii) if p's home server $s \neq s'$, then by liveness of the server-server Sync class, Sync over $\{s, s'\}$ occurs, updating $(q@s', posts)$ in c_s; (iii) Sync over $\{p, s\}$ occurs, delivering m to p.

Agent Autonomy. Post(m) over $\{p\}$ requires $(p@s, posts) \in c_p$, established at registration. Follow($q@s'$) over $\{p\}$ requires $c_p \neq \emptyset$ and $q@s'$ has no feed in c_p, enabled for unfollowed agents. ◀

▶ **Proposition 3.11.** *In the federated social network protocol, $E = Q$, $|E| = \infty$, and $|\Pi \setminus E| \geq 2$.*

Proof. If $P \cap Q = \emptyset$ (only clients, no servers), then the initial local state is $\emptyset$ and Register over $\{p, s\}$ requires $s \in Q$, which is unavailable. Hence no client can initialise (c_p remains $\emptyset$), and Post requires $p@s$ to have a feed in c_p, which never holds. Follow requires $c_p \neq \emptyset$, so it too is never enabled. No non-unary transition is possible. Hence Q is essential.

Minimality: for any server $s \in Q$, consider $P = \{q, s\}$ where $q \notin Q$. Register over $\{q, s\}$ is enabled—a non-unary transition. Hence $Q \setminus \{s\}$ does not suffice, and $E = Q$ is minimal. Every essential set must contain all of Q, since each non-unary transaction has a server as a participant; hence every essential set is infinite and $\kappa = \infty$. As the essential set Q has $|\Pi \setminus Q| \geq 2$, the protocol is federated. ◀

▶ **Proposition 3.13.** *The grassroots social network protocol satisfies Social Network Correctness.*

Proof. **Follower Correctness.** *Safety*: If $(q, posts) \in c_p$, then *posts* was obtained via direct Sync with q or transitively through other agents. The Sync transaction only updates to longer sequences, preserving prefix ordering. *Liveness*: If p follows q (q has a feed in c_p) and q posts m, then by liveness of the Sync class for $\{p, q\}$, Sync over $\{p, q\}$ eventually occurs, updating $(q, posts)$ in c_p to include m.

Agent Autonomy. Post(m) over $\{p\}$ requires $(p, s) \in c_p$, established by Initialise. Follow(q) over $\{p\}$ requires $p \neq q$ and q has no feed in c_p, enabled for unfollowed agents. ◀

▶ **Proposition 4.16.** *The grassroots social network protocol is grassroots under the original definition and grassroots$'$ under the interleaving-based definition.*

Proof. We first show the protocol is interactive (Definition 4.4). Let $\emptyset \subset P \subset P' \subseteq \Pi$ and let $c \in C(P')$ with $c/P \in C(P)$. Pick any $q \in P' \setminus P$ and any $p \in P$. From c, agent p can execute Initialise (if not yet initialised), then Follow(q), producing a configuration in which $(q, \varepsilon) \in c'_p$. No transaction in $\mathcal{F}(P)$ can produce a feed for an agent outside P, so $c'/P \notin C(P)$. Hence $\mathcal{F}$ is interactive, and therefore grassroots under the original definition.

The protocol is also grassroots$'$ under the interleaving-based definition (Definition 4.8). Obliviousness$'$ holds because the only transaction changing two local states is Sync, which requires that r have a feed in both c_p and c_q—and in an interleaving of runs of disjoint groups P and P', no agent in P can have a feed for an agent in P' (since Follow only adds feeds for agents named in its argument, and Sync only propagates existing feeds). For interactivity$'$, let $Q \subset \Pi$, let $P, P' \subseteq Q$ be disjoint and nonempty, let $\hat{r}$ be a prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P', and pick $p \in P$ and $q \in P'$. Extend $\hat{r}$ by whichever of Initialise, Follow and Post p and q have not already taken, so that each follows the other and each has posted, and then by Sync over $\{p, q\}$. That Sync updates p's feed for q and q's feed for p, so it changes the local states of both; and p's state then holds a feed for $q \neq p$, which is not a local state of $\mathcal{F}(\{p\})$, so its restriction to $\{p\}$ is not a transition of $\mathcal{F}(\{p\})$. It is therefore an interaction between p and q, hence between P and P'. Extending further by taking every enabled Sync class in turn gives a correct run of $\mathcal{F}(Q)$ having $\hat{r}$ as a prefix. By Theorem 4.9, all agents but one are essential. ◀

▶ **Proposition 3.14.** *In the grassroots social network protocol, $E = \Pi \setminus \{p\}$ for some $p \in \Pi$.*

Proof. By Proposition 4.16, the protocol is grassroots under both definitions. By Theorems 4.5 and 4.9, all agents but one are essential, so $E = \Pi \setminus \{p\}$ for some $p \in \Pi$. ◀

▶ **Proposition 4.12.** *Follow Only is interactive and not interactive′, and Paired Counter is interactive′ and not interactive. Hence neither definition implies the other.*

Proof. **Follow Only is interactive.** Let $\emptyset \subset P \subset P' \subseteq \Pi$ and $c \in C(P')$ with $c/P \in C(P)$, and take $p \in P$ and $q \in P' \setminus P$. As $c/P \in C(P)$ we have $F_p \subseteq P$, so $q \notin F_p$ and Follow(q) over $\{p\}$ is enabled at c; it yields c' with $q \in F'_p$, so $c'/P \notin C(P)$.

Follow Only is not interactive′. Its one transaction has a single active participant, so every transition of every run changes the local state of exactly one agent and none is an interaction between any two disjoint sets. The empty prefix of a correct run therefore contains no interaction and is a prefix of no run that contains one.

Paired Counter is interactive′. Let $Q \subset \Pi$, let $P, P' \subseteq Q$ be disjoint and nonempty, let $\hat{r}$ be a prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P', and take $p \in P$ and $q \in P'$. Tick over $\{p, q\}$ is enabled at every configuration, and extending $\hat{r}$ by it gives a correct run, as Λ is empty. It changes the local states of both p and q, and $\mathcal{F}(\{p\})$ has no transitions, as Tick has two participants; so its restriction to $\{p\}$ is not a transition of $\mathcal{F}(\{p\})$ and the Tick is an interaction between p and q, hence between P and P'.

Paired Counter is not interactive. No transaction changes the second component of a local state, and the first ranges over $\mathbb{N}$ for every population, so $c/P \in C(P)$ implies $c'/P \in C(P)$ for every computation $c \xrightarrow{*} c'$. Taking any $\emptyset \subset P \subset P' \subseteq \Pi$ and any such c therefore refutes interactivity. ◀

▶ **Proposition 4.15.** *In each of Silenced Speech and Entrenched Leader all agents but one are essential, and neither is interactive nor interactive′.*

Proof. **All agents but one essential.** In both protocols the binary transaction—Speak, Meet—is enabled at the initial configuration for any two agents, where $v = \mathit{open}$, $\ell = \bot$ and $M = \emptyset$. So a set E omitting two agents $p \neq q$ is not essential, as the run of $\mathcal{F}(\{p, q\})$ taking that transaction has a binary transition. Hence every essential set omits at most one agent. And $E = \Pi \setminus \{p\}$ is essential: $P \cap E = \emptyset$ gives $P \subseteq \{p\}$, so every transition of $\mathcal{F}(P)$ is unary; and it is minimal by the preceding sentence.

Not interactive. For Silenced Speech, let $\emptyset \subset P \subset P' \subseteq \Pi$ and let $c \in C(P')$ have $v_a = \mathit{silent}$ and $M_a = \emptyset$ for every $a \in P'$. Then $c/P \in C(P)$, and no transaction is enabled at c, as Speak requires two open agents and Silence one. So $c \xrightarrow{*} c'$ implies $c' = c$ and $c'/P \in C(P)$.

For Entrenched Leader, let $\emptyset \subset P \subset P' \subseteq \Pi$, fix $\ell \in P$, and let $c \in C(P')$ have $\ell_p = \ell$ and $M_p = P$ for every $p \in P$, and $\ell_a = a$ and $M_a = \emptyset$ for every $a \in P' \setminus P$. Then $c/P \in C(P)$. Meet and Elect require $\bot$, which no agent has; Relay over $\{a, b\}$ requires $\ell_a = b$ with $b \neq a$, which holds only for $a \in P$ and $b = \ell$, where $\ell \in M_a$ already. So nothing is enabled at c and $c'/P \in C(P)$ for every computation from c. Neither protocol is therefore interactive.

Not interactive′. For Silenced Speech, take any $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, and the prefix of a run of $\mathcal{F}(Q)$ in which every agent of Q takes Silence. It is a prefix of a correct run, as Λ is empty; every one of its transitions is unary, so none is an interaction between P and P'; and nothing is enabled at its last configuration, so no extension of it contains one.

For Entrenched Leader, take disjoint nonempty $P, P' \subset \Pi$, an agent $\ell \notin P \cup P'$, and $Q := P \cup P' \cup \{\ell\}$, and the prefix of a run of $\mathcal{F}(Q)$ in which every agent of Q elects ℓ. Each of its transitions changes one local state, so none is an interaction between P and P'. In

any extension, Meet and Elect are disabled and every Relay is over $\{a, \ell\}$ for some a, so no transition changes the local state of an agent of P and of an agent of P', and none is an interaction between them. Neither protocol is therefore interactive′. ◀

B Additional Global Platforms

Beyond social networks, the four-class characterisation applies to diverse global platforms. Formal classification requires a multiagent atomic transactions specification; most practical platforms lack one. The grassroots platforms cited below have been formally specified and proven grassroots, hence by Theorems 4.5 and 4.9 have all agents but one essential. The other classifications are informal: we anticipate any reasonable specification will confirm them.

Currencies and payment systems. Fiat currencies and Central Bank Digital Currencies (CBDCs) are centralised under government control. PayPal and Venmo similarly have single controlling servers. Bitcoin [40] and Ethereum [13, 64] exemplify decentralised systems with bootstrap nodes. Algorand [25] is likewise decentralised: nodes join through a published set of relays, and its peer-to-peer mode discovers peers through DNS seeds and a distributed hash table [2]. Ripple [50] operates as federated with designated validators. Grassroots coins and bonds [52, 57, 56] enable communities to bootstrap local economies from trust relationships.

"Sharing economy" platforms. Uber and Airbnb are centralised with corporate control. OpenBazaar [42] attempted decentralised peer-to-peer commerce using blockchain but ceased operations. Platform cooperatives [49] propose federated models where worker-owned nodes coordinate services. Grassroots digital cooperatives [52] would enable direct peer-to-peer resource sharing—transportation, housing, labour.

Content distribution. YouTube and Netflix are centralised. IPFS [7] uses decentralised DHT with bootstrap nodes. PeerTube [23] federates video hosting across servers. BitTorrent [17] is decentralised: its original version relies on a tracker and its trackerless version on DHT boot nodes.

Messaging. WhatsApp and Telegram are centralised. Signal is centralised; its sealed-sender mechanism [34] is a privacy feature within the centralised architecture. Matrix [37] and XMPP [48] federate across home servers. Scuttlebutt [59] and Briar [11] are grassroots with peer-to-peer messaging.

Computation. Cloud providers (AWS, Azure) are centralised. Ethereum provides decentralised computation [13, 64]. Grid computing [22] federates across institutions. Grassroots Logic Programs [55] is a concurrent logic programming language whose transactions-based semantics yields a grassroots protocol.

Governance. Existing voting platforms are typically centralised (Change.org). DAOs use decentralised blockchain voting [62]. Grassroots democratic federations [58, 57] support constitutional governance through community formation, sortition-based assembly selection, and multi-level federation.

AI and machine learning. OpenAI, Google AI, and Anthropic operate centralised platforms with models hosted exclusively in provider-controlled datacenters. Bittensor [9] achieves decentralisation through blockchain-based coordination with validators and miners. Petals [10] approaches grassroots with peer-to-peer inference where volunteers host model shards without central coordination.